

01 ¿Qué es la normalización?

Es el proceso de **organizar las tablas y sus columnas** para reducir la redundancia (datos repetidos) y eliminar las **anomalías** al insertar, actualizar o borrar registros. Fue propuesta por E. F. Codd (1970) dentro del modelo relacional.

- ⚠ Inserción**
No se puede guardar un dato sin registrar otro. Ej.: no existe el cliente sin su compra.
- ⚠ Actualización**
Un cambio debe repetirse en muchas filas. Ej.: cambiar el precio de un producto en cada venta.
- ⚠ Borrado**
Al eliminar un registro se pierde información. Ej.: borrar la venta elimina los datos del producto.

02 1FN — Primera Forma Normal

Regla: cada celda debe contener un **valor único y atómico**; no se permiten listas ni grupos repetidos (varios teléfonos en una sola columna).

✗ ANTES (viola 1FN)

id_cliente	nombre	telefono
1	Ana López	771-100-1111, 771-100-2222
2	Luis Pérez	771-100-3333



✗ Una celda con dos teléfonos separados por coma: grupo repetido.

✓ DESPUÉS (cumple 1FN)

id_cliente	nombre	telefono
1	Ana López	771-100-1111
1	Ana López	771-100-2222
2	Luis Pérez	771-100-3333

✓ Cada fila tiene un único valor; la clave pasa a ser (id_cliente, telefono).

03 2FN — Segunda Forma Normal

Regla: cumple 1FN y además cada atributo **no clave** depende de **TODA** la clave primaria (sin dependencias parciales). Aplica solo a claves compuestas.

✗ ANTES — dependencia parcial

id_venta	id_producto	cantidad	nombre_producto
1	5	2	Producto 5
2	5	1	Producto 5



✗ **nombre_producto** depende solo de **id_producto**, no de la clave completa (**id_venta, id_producto**). En **comercial_db**, el precio se guarda en **detalle_ventas** porque es el precio de venta del momento.

✓ DESPUÉS — dos tablas

id_producto	nombre	precio
5	Producto 5	65.00

id_venta	id_producto	cantidad
1	5	2
2	5	1

✓ **productos** guarda el catálogo; **detalle_ventas** solo la venta.

04 3FN — Tercera Forma Normal

Regla: cumple 2FN y no hay **dependencias transitivas**: los atributos no clave dependen solo de la clave, no de otro atributo no clave.

✗ ANTES — dependencia transitiva

id_cliente	nombre	id_ciudad	nombre_ciudad
1	Ana López	14	Pachuca



✗ **nombre_ciudad** depende de **id_ciudad**, que no es clave → transitiva.

✓ DESPUÉS — tabla ciudades

id_cliente	nombre	id_ciudad
1	Ana López	14

id_ciudad	nombre	id_estado
14	Pachuca	18

✓ Exactamente el diseño de **comercial_db**: **ciudades** e **id_ciudad** como FK en **clientes**.

05 Esquema normalizado — SQL en SQL Server

```
USE comercial_db;
GO

CREATE TABLE clientes
(
    id_cliente      INT IDENTITY(1,1) NOT NULL,
    nombre          VARCHAR(60) NOT NULL,
    sexo            CHAR(1),
    id_ciudad       INT NOT NULL,

    CONSTRAINT pk_clientes PRIMARY KEY(id_cliente),
    CONSTRAINT fk_clientes_ciudades
        FOREIGN KEY(id_ciudad) REFERENCES ciudades(id_ciudad)
);
```

Conclusión: un esquema normalizado (1FN, 2FN, 3FN) reduce la redundancia, elimina las anomalías y hace la base de datos **más fácil de mantener y consultar**: la integridad de los datos se vuelve responsabilidad del diseño, no de quien escribe las consultas.

Fuentes:

- Microsoft Learn — SQL Server: conceptos de bases de datos relacionales y normalización.
- W3Schools — SQL Tutorial: Database Normalization.
- Silberschatz, Korth y Sudarshan — Fundamentos de Bases de Datos (cap. sobre diseño relacional).